COMP.SE.140 – Docker-compose and microservices hands-on

1. Background

The purpose of this exercise is to get hands-on experience with containerized microservices, container networking, persistent storage, and interaction between services through REST APIs. The assignment is a simplified version of a multi-service system that consists of a gateway service that handles external requests, a secondary worker service, and a separate storage service with persistence. Further, it covers the topics such as cloud computing, virtualization, containers, and microservices.

2. Platform Information

The application was developed and tested on a laptop with an Intel i5 processor and 8 GB of RAM. The operating system was Windows 11 Home and the development environment used was Visual Studio Code with Docker Desktop installed on the machine.

Following table illustrates the different versions of software used to implement and test the system in local environment.

Software/ Tool	Version Number
Docker Client	27.4.0
Docker Desktop	4.37.1
Docker Compose	v2.31.0

Following table illustrates the different versions of software used for 3 different services in the given system.

Service	Base Image	Container OS	Runtime / Programming Language
Service 1 (Gateway)	node:18-alpine	Alpine Linux	Node.js v18.x LTS
Service 2 (Worker)	python:3.11-slim	Debian Slim	Python v3.11.x
Storage (Logger)	node:18-alpine	Alpine Linux	Node.js v18.x LTS

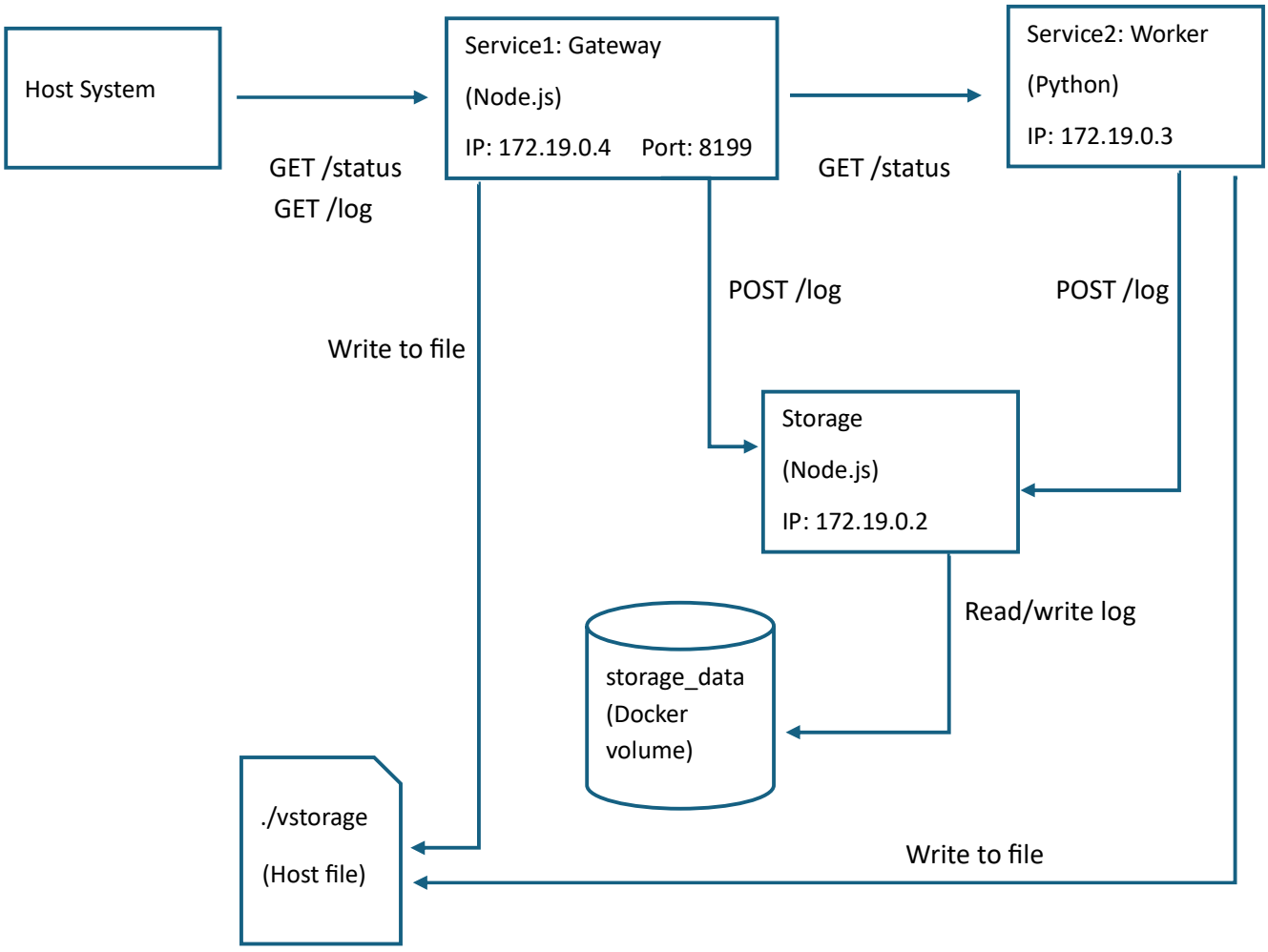
One of the main takeaways is that containerization abstracts away the platform differences. Even though the development was carried out on a Windows host using docker desktop, the containers run in a consistent Linux-based environment. This reflects portability and consistency of containerized applications across different operating systems.

3. System Architecture

The architecture consists of three microservices:

1. Service1 (gateway)
2. Service2 (worker)
3. Storage (logging service)

Following diagram shows the overall system architecture and the communication flow.



1. Service1 – Node.js (Express)
    - Role: The only service accessible from outside the container network (localhost:8199).
    - Responsibilities:
        - Collects system uptime and free disk space.
        - Stores logs in:
            - Shared host-mounted file volume vstorage
            - Storage service via REST calls
        - Forwards incoming /status requests to Service2 and aggregates results.
    - Endpoints:
        - GET /status
            - Analyzes uptime and free disk space
            - Appends a new log entry to both vstorage and the Storage service
            - Forwards the request to Service2 and combines both records in the response (as text/plain)
        - GET /log
            - Forwards the request to the Storage service and returns all stored logs (as text/plain)

2. Service2 – Python (Flask)
    - Role: Internal service, not exposed outside the Docker network.
    - Responsibilities:
        - Collects system uptime and free disk space.
        - Stores logs in:
            - Shared host-mounted file volume vstorage
            - Storage service via REST calls
        - Returns its status record to Service1.
    - Endpoints:
        - GET /status

        - Analyzes uptime and free disk space
        - Appends a new log entry to both vstorage and the Storage service
        - Returns its own status record as text/plain

3. Storage – Node.js (Express)
    - Role: Dedicated log persistence service.
    - Storage Method:
        - Uses a named Docker volume storage_data mounted at /data/log.txt to store logs persistently.
    - Endpoints:
        - POST /log - Append new log entry (as text/plain)
        - GET /log - Retrieve all stored logs (as text/plain)

4. vstorage – Shared Host-Mounted Volume
    - A learning-purpose persistent storage method.
    - Implemented by mounting the host file ./vstorage into both Service1 and Service2 at /vstorage.
    - Both services append log entries to this shared file for every /status request.

5. Networking
    - All three services are connected to a single Docker network: appnet.
    - Docker assigns each container a private IP (e.g., 172.19.0.2, 172.19.0.3, 172.19.0.4) within appnet.
    - Service1 is the only service that publishes a host port (8199), making it the gateway for external clients.
    - Service2 and Storage are accessible only inside the Docker network.

4. Analysis of Status Records

1. Measurements implemented:
    - **Uptime**: Shows how long the container itself has been running after start.
        - Service1: Read the container's app start time using process.uptime().
        - Service2: Subtracting the start time from the current time gives the elapsed time since the process started (time.time() - ps_start).
    - **Disk space**: The amount of free disk space available on the root file system.
        - Service1: Measured from / root filesystem in MB using Node.js' built-in child_process module.
        - Service2: Measured from / root filesystem in MB using shutil library.

In a DevOps context, uptime allows us to detect restarts or failures of individual services. Disk space is measured using the root filesystem inside the container (df -m /). However, this value does not reflect the space available to the individual service but rather the free disk space of the host system's filesystem used by Docker.

5. Persistent Storage

Two different storage approaches were implemented:

1. Host file (./vstorage)

    Using this method, logs are appended directly into a file on the host filesystem. This makes it transparent and easy to inspect using simple cat ./vstorage. However, this approach is considered poor practice in real deployments because it couples the container to the host operating system, introduces cross-platform inconsistencies (especially between Windows and Linux), and may cause permission issues or security issues.

2. Container-managed Docker volume (storage_data)

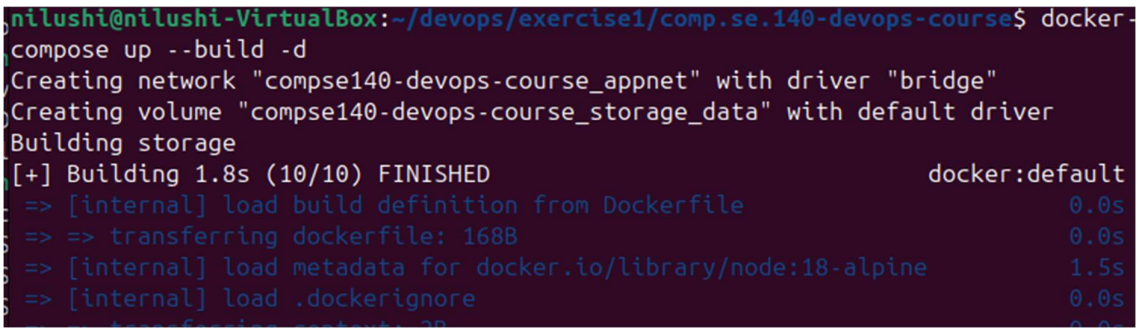
   This method is used by the Storage service. This is the recommended approach in real-world systems because it abstracts storage away from the host and keeps the data within the container runtime. Volumes are portable, more secure, and easier to scale. The drawback is that they are less transparent, as the logs are not directly visible from the host without using docker exec or volume inspection commands.

Overall, the exercise shows that while the bind-mount method is simpler for demonstration, the volume-based approach is the better approach in terms of maintainability, security, and cloud-readiness.

6. How to Run the Application

1. Clone the repository using command:

   git clone -b exercise1 https://github.com/silshara/comp.se.140-devops-course.git
2. cd comp.se.140-devops-course
3. docker-compose up --build -d
4. wait until services are started
5. curl localhost:8199/status (in a new terminal window, if detached mode -d is not used with docker-compose up)
6. curl localhost:8199/log
7. cat ./vstorage
8. docker-compose down



```
 => exporting to image                                                    0.0s
 => => exporting layers                                                   0.0s
 => => writing image sha256:915da6e0f527f3afa86347734a0c37175ea9c08809eac  0.0s
 => => naming to docker.io/library/microservice_gateway:latest            0.0s
Creating compse140-devops-course_storage_1 ... done
Creating compse140-devops-course_service2_1 ... done
Creating compse140-devops-course_service1_1 ... done
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ curl lo
calhost:8199/status
2025-09-28T22:19:35Z, uptime 0.01 hours, free disk in root: 18202 MBytes
2025-09-28T22:19:35Z, uptime 0.01 hours, free disk in root: 18202 Mbytesnilushi@
nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ curl localhost:
8199/log
2025-09-28T22:19:35Z, uptime 0.01 hours, free disk in root: 18202 MBytes
2025-09-28T22:19:35Z, uptime 0.01 hours, free disk in root: 18202 Mbytes
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$
```

7. Instructions for Cleanup

To reset the logs, both storage solutions must be cleaned.

1. Mounted file (./vstorage),
   - On Linux this can be done with: > ./vstorage

```
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ > ./vst
orage
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ cat ./v
storage
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ docker
```

   - Additionally, an api endpoint is also created under service1 to clear the vstorage.

     curl -X DELETE localhost:8199/vstorage

2. Storage service volume (storage_data)

   There are few cleanup options based on the different needs.

   - Use the following commands to remove the named volume after docker-compose down is already executed. (compse140-devops-course_storage_data is the volume name)
     1. docker volume ls

        Verify that the given volume name is correct, using ls command. If not, replace with the correct name in the following command.

     2. docker volume rm compse140-devops-course_storage_data

```
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ docker
volume ls
DRIVER    VOLUME NAME
local     compse140-devops-course_storage_data
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ docker
volume rm compse140-devops-course_storage_data
compse140-devops-course_storage_data
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ docker
volume ls
DRIVER    VOLUME NAME
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ docker
```

This will delete the named volume and all data inside it if no container is currently using it. If it is in use, Docker will prevent deletion.

- To remove the volume entirely while removing the containers:

  docker-compose down -v

  This removes all the containers and removes the volume, storage_data.

```
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$ docker-
compose down -v
Stopping compse140-devops-course_service1_1 ... done
Stopping compse140-devops-course_service2_1 ... done
Stopping compse140-devops-course_storage_1  ... done
Removing compse140-devops-course_service1_1 ... done
Removing compse140-devops-course_service2_1 ... done
Removing compse140-devops-course_storage_1  ... done
Removing network compse140-devops-course_appnet
Removing volume compse140-devops-course_storage_data
nilushi@nilushi-VirtualBox:~/devops/exercise1/comp.se.140-devops-course$
```

- Alternatively, if it is needed to keep the volume but clear the log file inside it, the following command can be used:

  curl -X DELETE http://localhost:8199/log

  This is an extra endpoint added to service1 for log deletion.

3. Optionally, to remove Docker Images,

docker image rm microservice_gateway microservice_worker microservice_storage

8. Key Problems and Difficulties

- Ensuring that only Service1 was accessible externally.

  This required careful use of Docker Compose, making sure that only Service1 had a ports mapping while Service2 and Storage remained internal using custom network.

- Working with persistent storage on Windows (vstorage)

  Commands like cat didn't work on Windows, so alternatives had to be used (Get-Content, Clear-Content).
- Additionally, PowerShell's built-in alias for curl caused some confusion as it returns truncated results unless .Content is accessed explicitly.

- Understanding resource metrics between containers and host.

  When checking the uptime and disk space for status requests, it was initially unclear whether the reported values correspond to the container itself or the underlying host VM.

9. Conclusion

This exercise was helpful to understand how cloud-native, containerized systems are designed and deployed. It tied together several key concepts such as virtualization, containers, and microservices.

By completing the exercise, I learned how containers provide both isolation and portability, how Docker networks create virtualized communication, and how persistent storage can be handled in different ways. I also gained experience in structuring microservices across multiple programming languages.